

Form Builder

CRM/ERP Platform Documentation

Form Builder

Forms allow collecting data from external (or internal) users without giving them CRM access. Submitted data is saved as `FormSubmission` rows and optionally as `Record` rows in a module.

Key Concepts

Concept	Description
Form	A configured data collection form with a public URL
Token	Unique URL token (e.g. <code>/f/abc123</code>) for the public form
Section	Visual grouping of fields within a form
FormField	A reference to a module <code>Field</code> , with optional customizations (label, placeholder, required override)
Submission	A submitted form response stored in <code>FormSubmission</code>
OCR Upload	Document scanning feature — uploads a PDF/image, Claude extracts field values
Submission Receipt	Printable confirmation page shown after successful submission

Form Builder UI

Located at `/forms/[id]/builder`. Tabs:

Tab	Key	Description
Fields	<code>fields</code>	Drag-and-drop field layout
Settings	<code>settings</code>	Form name, description, module binding
OCR Upload	<code>documents</code>	Configure document extraction
Submission Receipt	<code>ticketing</code>	Enable receipt + custom message
Permissions	<code>permissions</code>	Role-based access to the form
Sharing	<code>sharing</code>	User/dept/role sharing controls

Field Layout

- Fields can be dragged to reorder (dnd-kit sortable)
- Whole card is draggable; interactive elements (inputs, buttons) stop drag propagation with `onPointerDown={e => e.stopPropagation()}`
- Right-edge resize handle toggles field width between 50% and 100%
- Drop indicator line shows insertion point
- `BuilderDragCtx` React context shares `activeId` / `overFieldId` without prop drilling

Public Form (`/f/[token]`)

The public form page is server-rendered via `GET /public/forms/:token` . It:

1. Shows a landing page with OCR upload option (if OCR is configured)
2. Renders the form fields
3. Validates required fields client-side
4. Submits via `POST /public/forms/:token/submit`
5. Shows a **Submission Receipt** (if enabled) or a generic success message

Field Rendering

Each field type has a specific renderer:

- **RADIO**: Custom `<div role="radio">` components — visually styled, keyboard accessible
- **SELECT/DROPDOWN**: `<select>` or combobox

- **FILE/IMAGE:** Drag-and-drop uploader
- **SIGNATURE:** Canvas-based signature pad
- **DATE/DATETIME:** Date picker

Filled field indicator: Fields with a non-empty value show a green border (`border-emerald-300`) and a `CheckCircle2` icon.

OCR Document Extraction

Configured under the **OCR Upload** tab in the builder.

Configuration

Enable OCR and select which fields should be auto-filled from the document:

```
{
  "documents": {
    "enabled": true,
    "fieldIds": ["field-id-1", "field-id-2"]
  }
}
```

User Flow

1. Public form landing page shows the OCR upload area
2. User uploads a PDF or image
3. Frontend sends `POST /public/forms/:token/extract-document` with the file as `multipart/form-data`
4. Backend sends the image to Claude (`claude-sonnet-4-6`) with a prompt listing the fields
5. Claude returns a JSON of `{ fieldName: extractedValue }`
6. Frontend normalizes values (case-insensitive matching for RADIO/SELECT options)
7. Form fields are pre-filled; user can correct before submitting

Option Normalization (4-tier matching)

For RADIO/SELECT/DROPDOWN/STATUS/MULTI_SELECT fields, extracted values are matched against options:

1. Exact value match
2. Case-insensitive value match
3. Case-insensitive label match
4. Partial label match (either direction)

This handles cases where Claude returns `"Male"` but the option value is `"male"`.

Backend Prompt

The backend builds a field description list that tells Claude the allowed values:

```
- "gender" (Gender) [MUST be one of: "male", "female", "other"]
- "full_name" (Full Name)
```

Submission Receipt

When `settings.ticketing.enabled` is true, after submission the user sees a printable receipt instead of a generic success page.

Receipt Contains

- Application ID (auto-generated `APP-{timestamp}` or the `ticketNumber` from DB)
- Submission date and time
- Status: **Received**
- Table of all submitted field values (in form-field order)
- Custom message (from `settings.ticketing.message`)
- **Print Receipt** button (`window.print()`)
- **Submit Another Response** button (`window.location.reload()`)

Data Flow

The submission response from `POST /public/forms/:token/submit` includes:

```
{
  "submittedSnapshot": { "fieldName": "value", ... },
  "submittedAt": "2026-06-30T12:00:00Z",
```

```
"appId": "APP-1234567890"
}
```

The frontend stores this in state and renders the receipt.

Conditional Logic

Form fields support `conditionalLogic` — a JSON rule that shows/hides a field based on other field values:

```
{
  "action": "show",
  "conditions": [
    { "field": "employment_type", "operator": "equals", "value": "employed" }
  ]
}
```

The rule is evaluated client-side as the user fills in the form.

URL Parameters

FormFields can have a `urlParamKey`. If the public form URL includes `?fieldName=value`, the field is pre-filled with that value (and optionally made read-only).

Form Submissions

View all submissions for a form: `GET /forms/:id/submissions`.

Each submission stores:

- `data` : JSON blob of submitted values
- `ipAddress`, `userAgent` : For audit purposes
- `ticketNumber` : Auto-assigned reference number

Permissions

Form access is controlled by `FormPermission` rows. By default:

- All roles can view and submit
- Only admins can edit or manage the builder

Sharing settings (`Form.sharedUsers` , `Form.sharedDepts` , `Form.sharedRoles`) allow sharing with specific people.

Generating a Public Link

```
POST /api/v1/forms/:id/generate-token
```

Returns a `token` string. The public URL is `https://app.yourdomain.com/f/{token}` .

Revoke with `POST /api/v1/forms/:id/revoke-token` — the old URL immediately stops working.